

Optional<>

Esquema de la clase Optional<>

- **Concepto básico:** Optional es un contenedor que puede o no contener un valor no nulo
- **Propósito:** Evitar NullPointerException y hacer el código más expresivo
- **Métodos principales de creación:**
 - `Optional.of(valor)` - Crea un Optional con un valor (lanza excepción si es null)
 - `Optional.ofNullable(valor)` - Crea un Optional que puede ser vacío
 - `Optional.empty()` - Crea un Optional vacío
- **Métodos para verificar contenido:**
 - `isPresent()` - Devuelve true si contiene un valor
 - `isEmpty()` - Devuelve true si está vacío
- **Métodos para obtener valores:**
 - `get()` - Obtiene el valor (lanza excepción si está vacío)
 - `orElse(valorPorDefecto)` - Devuelve el valor o un valor por defecto
 - `orElseGet(supplier)` - Devuelve el valor o ejecuta una función
 - `orElseThrow()` - Devuelve el valor o lanza una excepción
- **Métodos funcionales:**
 - `map()` - Transforma el valor si está presente
 - `flatMap()` - Transforma y aplana Optional anidados
 - `filter()` - Filtra el valor según una condición
 - `ifPresent()` - Ejecuta una acción si el valor está presente

Explicación detallada

¿Qué es Optional?

Optional es una clase genérica introducida en Java 8 que actúa como un contenedor para un valor que puede estar presente o ausente. Su objetivo principal es proporcionar una forma más segura y expresiva de manejar valores que podrían ser null.

Creación de Optional

```
// Crear Optional con valor (no acepta null)
Optional<String> opt1 = Optional.of("Hola");

// Crear Optional que puede ser null
```

```
Optional<String> opt2 = Optional.ofNullable(null);

// Crear Optional vacío
Optional<String> opt3 = Optional.empty();
```

Verificar si contiene valor

```
Optional<String> optional = Optional.of("Mundo");

if (optional.isPresent()) {
    System.out.println("Contiene: " + optional.get());
}

// Java 11+
if (optional.isEmpty()) {
    System.out.println("Está vacío");
}
```

Obtener valores de forma segura

```
Optional<String> optional = Optional.ofNullable(getNombre());

// Opción 1: Valor por defecto
String nombre = optional.orElse("Desconocido");

// Opción 2: Valor calculado (solo se ejecuta si está vacío)
String nombre2 = optional.orElseGet(() -> obtenerNombrePorDefecto());

// Opción 3: Lanzar excepción personalizada
String nombre3 = optional.orElseThrow(() -> new IllegalArgumentException("Nombre no encontrado"));
```

Transformaciones con map y flatMap

```
Optional<String> nombre = Optional.of("Juan");

// map: transforma el valor
Optional<Integer> longitud = nombre.map(String::length);

// flatMap: para Optional anidados
Optional<String> apellido = obtenerApellido(nombre.get());
Optional<String> resultado = nombre.flatMap(n -> obtenerApellido(n));
```

Filtrado

```
Optional<Integer> edad = Optional.of(25);

// Solo mantiene el valor si cumple la condición
Optional<Integer> mayorEdad = edad.filter(e -> e >= 18);
```

Ejecutar acciones

```
Optional<String> mensaje = Optional.of("Hola");

// Ejecuta solo si está presente
mensaje.ifPresent(m -> System.out.println(m));

// Java 9+: ifPresentOrElse
mensaje.ifPresentOrElse(
    m -> System.out.println("Mensaje: " + m),
    () -> System.out.println("Sin mensaje")
);
```

Ejemplo práctico completo

```
public class EjemploOptional {

    public Optional<Usuario> buscarUsuario(int id) {
        // Simula búsqueda en base de datos
        Usuario usuario = db.find(id);
        return Optional.ofNullable(usuario);
    }

    public void procesarUsuario(int id) {
        buscarUsuario(id)
            .filter(u -> u.isActivo())
            .map(Usuario::getNombre)
            .map(String::toUpperCase)
            .ifPresentOrElse(
                nombre -> System.out.println("Usuario: " + nombre),
                () -> System.out.println("Usuario no encontrado o inactivo")
            );
    }
}
```

Buenas prácticas

- **NO uses get() sin verificar:** Siempre usa `orElse`, `orElseGet` u `orElseThrow`
- **NO uses Optional en campos de clase:** `Optional` está diseñado para valores de retorno
- **NO uses Optional.of() con valores que puedan ser null:** Usa `ofNullable()`
- **Encadena operaciones:** Aprovecha `map`, `filter` y `flatMap` para código más limpio
- **Prefiere orElseGet() sobre orElse():** `orElseGet` es perezoso y más eficiente

Beneficios de usar Optional

- Código más expresivo y legible
- Reduce `NullPointerException`
- Fuerza a considerar el caso de valor ausente
- Facilita la programación funcional en Java
- Mejora la API documentando valores opcionales